Assignment 1

Gradient Descent Based Optimization Using Auto-Differentiation

© Industrial Analytics Group
Dept. Civil and Industrial Engineering, Uppsala University

Contents

1 Introduction/Overview

1.1 The Importance of Optimization and Auto-differentiation

During this course, you will probably notice that we make a big deal about optimization problems, meaning finding local minima (or maxima) to functions of interest. You might even be inclined to initially think that we are obsessively focused on optimization methods in all shapes and forms. This is because we are - simply because optimization is the driver and corner stone underlying most modern technologies. Performing optimization is a core component of most of modern areas of engineering and computer science, including Artificial Intelligence, Machine Learning, Industrial Operations Research, Telecommunication, Autonomous Vehicle Navigation, and more.

In fact, **almost any problem and process you will face (as an engineer, data scientist, industrial analysis expert, or researcher) can be formulated as a task of finding a local minimum to a mathematical function**. Using such a "function perspective", how a human/agent or a system "acts" may often be described quantitatively as a multivariable input vector $\mathbf{x}$ and what is returned back/observed is a corresponding function value $f(\mathbf{x})$. Typically one wants to be very "efficient" in finding the optimal input $\mathbf{x}^*$ that gives back the smallest possible function value $f(\mathbf{x}^*)$. By efficient we mean using few function evaluations and/or actually finding the global rather than a local optimum.

One key challenge is to translate the practical real world "problem" of interest into a corresponding computational optimization problem. Our optimization journey begins here with the ***Gradient Decent Algorithm***, and how gradients can be calculated efficiently and conveniently for very complex functions using so called ***Auto-differentiation***.

Auto-differentiation is a quite old computational methodology that in recent years have gained enormous attention in the form of novel modern freely available implementations such as PyTorch and Tensorflow. The main reason for this recent development is that Auto-differentiation has become a key enabler of AI, where the tunable parameters of different Artificial Neural Networks (ANNs) are optimized using variations of the Gradient Descent Algorithm. Therfore, in order to fully understand what is behind the recent breakthroughs in AI, **one must study Gradient Descent Optimization Using Auto-differentiation**.

1.2 The Problem

Here you will learn about Gradient Descent, and some improved variants, as a powerful way of finding local minima of a differentiable mathematical function $f()$ that has a scalar input $x \in \mathbf{R}$ or a vector input $\mathbf{x} \in \mathbf{R^d}, d > 1$. Stated mathematically, the goal here is to learn about various ways of solving the minimization problem

$$\mathbf{x}^* = \arg\min_{\mathbf{x} \in \mathbf{R^d}} f(\mathbf{x})$$

where "arg min" is an abbreviation for saying that we want the argument (input) $\mathbf{x}^*$ that gives the smallest possible (minimum) function value $f(\mathbf{x}^*)$.

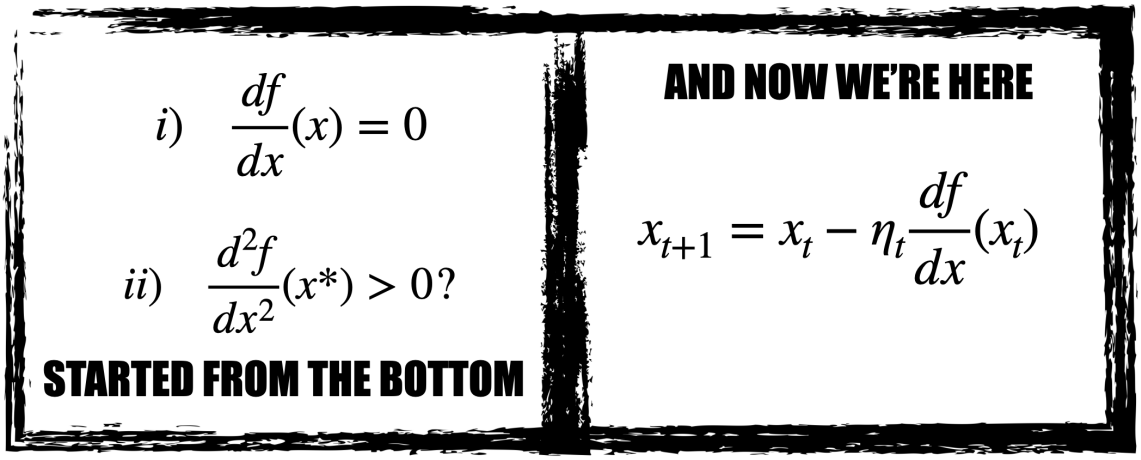


Figure 1: This assignment is about Gradient Decent, an iterative search method for finding local minima of functions that are differentiable. The core idea of Gradient Decent is to iteratively make small steps along the opposite direction of the gradient of the function one wants to minimize, beginning at some initial point x_0. The size of the current step is determined by the user defined parameter η, which often is referred to a "step size" or "step length", although this terminology is actually misleading - because the true step size is actually $\eta_t ||\nabla f(\mathbf{x}_t)||$.

1.3 Gradient Descent

When using the Gradient Descent Algorithm, the idea is to find the mimumum point $\mathbf{x}^*$ by moving in the opposite direction of the gradient vector

$$\frac{\partial f}{\partial \mathbf{x}} = \nabla f(\mathbf{x}) = (\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_d})^T.$$

This means that the ***the Gradient Decent update rule*** can be expressed as:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta_t \nabla f(\mathbf{x}_t). \tag{1}$$

The illustration in Fig. 1 shows how we in this assignment will move from the basic (bottom) idea in Calculus to find stationary points where the gradient (derivative) is zero, to an iterative numerical approach where we move in the opposite direction of the gradient in each step, in an iterative manner. Thus, in this assignment you will get the task to solve a set of small and basic optimization problems (on the theme "...write some code that minimizes this function $f(x)$"). There will be a mixture of both pen and paper tasks as well as tasks where you have to implement algorithms and solutions in Python, using a few very recent and powerful package called **PyTorch**. The objective functions that you will often be dealing may look similar to that in Fig. 2, where there is the possibility that more than one minimum or maximum exist within an search interval of interest.

1.4 The step length parameter η_t

In the context of Gradient Descent. the user defined parameter η_t is often called "step length". It is often a time varying parameter that determines how large the step in the opposite direction is when computing a new candidate solution, $\mathbf{x}_t + 1$. Unfortunately, the

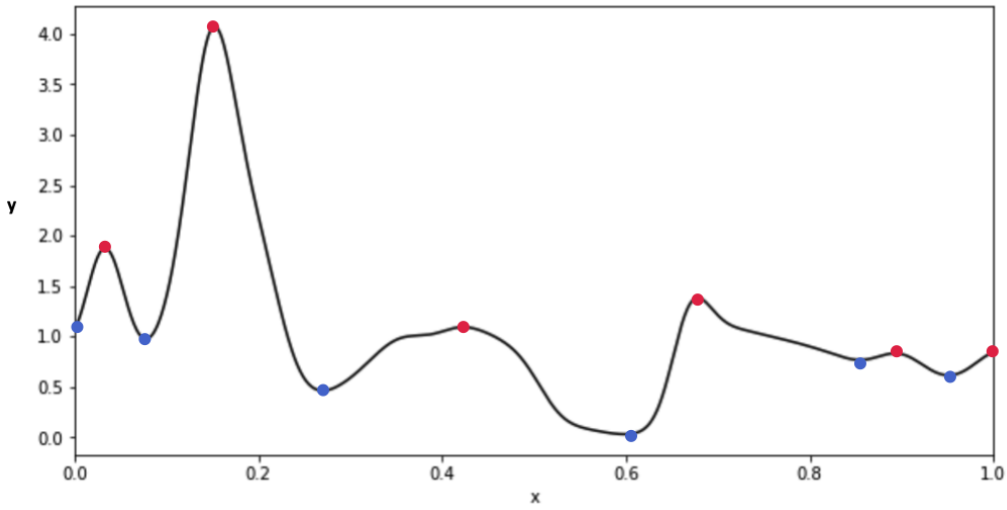


Figure 2: A single variable function with several minima (blue dots) and maxima (red dots).

term "step length" is actually misleading - because the true step size is actually $\eta_t ||\nabla f(\mathbf{x}_t)||$.

1.5 Solving maximization problems

Without loss of generality, here we are studying the problem of finding minimum points $\mathbf{x}^*$ that locally minimizes $f(\mathbf{x})$. If we instead would like to find maxima of some function $g(\mathbf{x})$ of interest, they can be found by searching for the minima of a function $f(\mathbf{x})$ defined as $f(\mathbf{x}) = -g(\mathbf{x})$. In other words

$$x^* = \arg\max_x g(\mathbf{x}) = \arg\min_x -g(\mathbf{x})$$

1.6 The gradient direction

Remember that the "gradient" of a multivariable function $f(\mathbf{x})$ may be regarded as a generalization of the "derivative" concept used for single variable functions. One of the main outstanding and useful properties is that **the gradient always points in the direction of maximum growth** in the input space. This mean the direction in which the function $f(\mathbf{x})$ grows most rapidly - the direction of greatest/steepest **ascent**. Therefore, by moving a small step along the opposite direction of the gradient, we have a good chance to get to a new position $\mathbf{x}_{\text{new}}$ with a smaller function value $f(\mathbf{x}_{\text{new}})$.

1.7 Goals

After completing this assignment, you should be have gained the skills to:

1. find local minima to functions $f(\mathbf{x})$ by paper and pen calculations, including the use of the chain rule. (Task A)
2. perform Gradient Descent using ***analytical (symbolic) expressions*** for the gradients. (Task B)

3. perform numerical calculations of gradients using ***auto-differentiation*** implemented in the generic but neural network focused ***PyTorch*** framework. (Task C)

4. perform Gradient Descent for some concrete optimization problems using ***auto-differentiation*** implemented in the generic but neural network focused ***PyTorch*** framework. (Task D and E)

NOTE: We don't believe in teaching you things "just because". We promise that everything that we teach will also be something you will need to use for solving and understanding this and later assignments. We also promise you that these tools will be quite generic, meaning very universal methods invaluable to have in your mental and practical toolbox.

1.8 Organization

Following the list of goals presented above, this assignment is divided into the following tasks.

NOTE: For Task A, there are no mandatory exercises, but for Tasks B-D the associated exercises are mandatory.:

Task A: Example problems where you should remind yourself how to perform analytical differentiation of the objective functions $f(\mathbf{x})$ of interest in order to find stationary points and/or points that minimize f.

Task B: Numerical Gradient Descent using analytical expressions for the desired gradients.

Task C: Calculating partial derivatives using ***PyTorch*** based ***Auto-differentiation***.

Task D and E: Numerical Gradient Descent using ***PyTorch*** based ***Auto-differentiation***.

1.9 Python Programming MUST be performed in Google Colab

This assignment MUST be performed by means of the freely available Google Colab. (https://colab.research.google.com/notebooks/intro.ipynb), which offers a notebook in the cloud, which is more or less identical to the Jupyter notebook (for those who are familiar with this notebook). For a 3 minutes long video introduction to Colab see: https://youtu.be/inN8seMm7UI. In order to use Colab, all you need is a personal gmail (Google) account as such an account has to be specified when starting Colab. You may easily create a personal gmail account at www.google.com. NOTE: If you do not want to use your current gmail account (if you already have one), you can create a new gmail account to be used only during this course.

Once you are inside the Colab notebook, first rename the name of your current notebook by clicking on the text on the top line and then changing the name into something like `Assignment1.ipynb`. Note: Your Notebook files will be stored automatically on your Google Drive, under a subfolder called `Colab Notebooks`!

MOUNT YOUR PERSONAL GOOGLE DRIVE TO COLAB:
You should mount your personal Google Drive to Colab using the following code:

```
#Mount your personal Google Drive to colab
from google.colab import drive
drive.mount('/content/drive')
import os
os.chdir('drive')
os.chdir('MyDrive')
```

Note: This will require an small additional intermediate step where you have to approve the mounting (this is straightforward, just follow the instructions that automatically appear).
Then you can check that everything looks as expected by running the following two lines of code (in separate windows):

```
os.getcwd() #Get current working directory

os.listdir() #list content of current directory
```

CREATE A COURSE SPECIFIC SUBFOLDER CALLED AI_for_IA:
You should also create a subfolder called AI_for_IA and make it your current working directory:

```
if not os.path.isdir("AI_for_IA"):  # create subfolder AI_for_IA if it does
  os.mkdir("AI_for_IA")             # not already exist
os.chdir("AI_for_IA") # move into the new subfolder => current working directory
os.getcwd()    # check which is the current working directory
```

In summary, this code makes subfolder AI_for_IA on your Google Drive your working directory.

NOTES:
(i) If you get a message that the drive already is mounted (or similar), select to "Disconnect and reset runtime" under menu"Runtime".
(ii) Colab save newly created notebooks in its own folder Notebooks. You may "manually" move them from there into your own folder "AI_for_IA", if you want.

RECOMMENDED VIDEOS TO WATCH:
Video about Colab and its notebook: https://youtu.be/inN8seMm7UI
Video about Python programming: https://www.youtube.com/watch?v=N4mEzFDjqtA

2 Task A: By-hand Differentiation of Objective Functions

Here you will warm up and repeat optimization the way your grandma and grandpa taught you when life was simple. More specifically, here you will refresh and repeat how to perform optimization of simple functions by determining the derivative and setting it to zero, to then find a solution to the resulting equation, and then check if it is a minimum by calculating the 2nd derivative of the function, plugging in the candidate solution and check for the sign. This is the type of optimization method you were taught during your first course in Calculus. This will serve as our starting point because we need revisit our familiar territory before we venture into more realistic, useful and efficient methods of finding optima to objective functions.

2.1 Theory

Gradient Descent optimization is presented below in the form of Grandma's algorithm (Algorithm 1) covering the single variable case and Grandpa's algorithm (Algorithm 2) covering the 2-dimensional case (and therefore in practice also the general d-dimensional/multivariable case where $d > 2$).

Algorithm 1 Grandma's "Set 1st derivative to zero and check with the 2nd derivative test" recipe

Problem: Find stationary points of a function $f(x)$

0. Check if function f(x) is differentiable. If not, stop here as there is no way forward in this case.
1. Calculate expression of the derivative $\frac{df}{dx}$
2. Set the derivative equal to zero

$$\frac{df}{dx}(x) = 0$$

3. Solve the above equation and get candidate solutions x^*
4. Check if a candidate solution is actually a minimum by the 2nd derivative test.

$$\frac{d^2f}{dx^2}(x^*) > 0 \implies \text{local minimum}$$

$$\frac{d^2f}{dx^2}(x^*) < 0 \implies \text{local maximum}$$

$$\frac{d^2f}{dx^2}(x*) = 0 \implies \text{"inflection point"}$$

As you will experience below, it is certainly doable to use both of them for small and non-complicated functions that are well-behaved. However, once our objective functions become more complex and have multidimensional inputs, then "the Grandma & Grandpa" method becomes very frustrating and inefficient to use. The following problems will act as a condensed journey towards increasing difficulty and experiencing the accompanying frustration. This will be a good theoretical reminder and also help you appreciate the computational tools to be discussed later, that can help you with these tasks.

2.2 Exercises

NOTE: The following exercises are NOT MANDATORY but highly recommended. The exercises are in order to convince yourself that you have understood and remember. If you struggle here, take

Algorithm 2 Grandpa's"Set all partial derivatives to zero and check Hessian of f(x,y)" recipe

Problem: Find stationary points of a function $f(x, y)$

0. Check if function $f(x, y)$ is differentiable. If not, stop, there is no way forward in this case.
1. Calculate the partial derivatives $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$
2. Set both the partial derivatives equal to zero

$$\frac{\partial f}{\partial x}(x, y) = 0$$

$$\frac{\partial f}{\partial y}(x, y) = 0$$

3. Solve the above system of equations and get candidate solutions (x^*, y^*)
4. Calculate all possible 2nd order partial derivatives to create the ***Hessian Matrix***:

$$H(x, y) = \begin{bmatrix} \frac{\partial^2 f}{\partial x \partial x} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y \partial y} \end{bmatrix}$$

5. Check if candidate solution is actually a local minimum, a local maximum or a ***saddle point*** by checking if the eigenvalues of the Hessian matrix at the candidate solution (x^*, y^*) i) all are real and negative $\implies$ local maximum, ii) all are real and positive $\implies$ local minimum, or iii) have mixed signs $\implies$ saddle point.

it as a warning signal to brush-up on your calculus knowledge/memory.

Exercise A.1
Find the global minimum of f and "show" that it is indeed a true minimum using the sign of the 2nd derivative at your candidate solution.

$$f(x) = (x - 5)^2 \tag{2}$$

.

Exercise A.2
Find the global minimum of f and "show" that it is indeed a true minimum.

$$f(x) = -e^{-(x-2)^2} \tag{3}$$

Exercise A.3
Find the global minimum of f and "show" that it is indeed a true minimum by means of the Hessian.

$$f(x, y) = 3 \cdot (x - 2)^2 + 4 \cdot (y - 1)^2 \tag{4}$$

Exercise A.4
Determine the gradient of the function f at the point $(x_o, y_o) = (1, 3)$.

$$f(x, y) = (y - 2)^2(x - 5)^2 + (x - 5)^2 + (y - 2)^2 \tag{5}$$

Exercise A.5
Try to find the global minimum of f.

$$f(x, y) = (y - 2)^2(x - 5)^2 + (x - 5)^2 + (y - 2)^2 \tag{6}$$

You will find that it is more cumbersome. Try until you feel a bit sweaty.

3 Task B: Gradient Descent based on NumPy and Manual Differentiation

Here we will deal with a numerical approach to finding local optima, which bypasses step 2 and 3 in the Grandma's optimization algorithm (Algorithm 1).

3.1 The Gradient Descent Algorithm

The famous ***Gradient Decent*** algorithm, presented here as Algorithm 3, is named like this simply because its creates moves in the current direction along which the objective function f has its largest descent. Therefore it is also often called the ***Steepest Decent*** Algorithm.

Notably, the standard form of Gradient Descent is not based on a normalized gradient, which has unit length. Therefore the search direction actually has a magnitude, not only a direction. This in turn means that the scalar multiplication factor η_t used cannot be interpreted as a true step length. This is simply because the true step length $||\mathbf{x}_{t+1} - \mathbf{x}_t||$ actually equals $\eta_t||\nabla f(\mathbf{x}_t)||$.

Algorithm 3 The Gradient Decent Algorithm

First: Specify some initial guess at the solution (does not have to be a very good guess) $\mathbf{x}_0$.

For t=0,1,2,3,4...

1. Do you find $\mathbf{x}_t$ sufficiently good ("optimal") for your problem? If yes, then stop. If not, then:

2. Calculate the gradient of the objective function at $\mathbf{x}_t$, meaning compute $\nabla f(\mathbf{x}_t)$. This gradient vector is parallel to the desired maximally descending search direction, but it points in the opposite direction. Therefore use the **negative** (!!) of this vector to prepare for a move in the desired search direction.

3. Choose a scalar multiplication factor η_t, which may change from one iteration to the next.

4. Update your next solution candidate by moving along the desired search direction:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta_t \nabla f(\mathbf{x}_t)$$

3.2 The Gradient Descent Algorithm in terms of coding

So what does the pseudo-code in Algorithm 3 tell us when we are tasked with finding a minimum of a certain function? Practically, here is what we need to do:

1. code up the function of interest we want to minimize.
2. code up the gradient of the function want to minimize,
3. code up an iterative loop with the update equation

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta_t \nabla f(\mathbf{x}_t).$$

,

4. Update the value of η_t according to some user defined procedure, or keep it constant.

5. incorporate one or several ***termination (stopping)*** criteria to ensure that the iterative loop stops at some point. For example, it makes sense to stop the iterations when the magnitude of the gradient is smaller than some threshold τ. Mathematically, this ccondition can be expressed as $||\nabla f(\mathbf{x}_t)|| < \tau$.

3.3 Exercises

NOTE: Before starting with problem B.3 below, you should make sure you have some basic knowledge about how to use the NumPy package for numerical calculations.

Exercise B.1 - Step size:
The size of the current step is determined by the user defined parameter η in Eq. (1), which often is referred to a "step size" or "step length". However, this terminology is actually misleading: Explain the fact that the true step size is not η_t but instead $\eta_t||\nabla f(\mathbf{x}_t)||$.

Exercise B.2 - Stopping Criteria: Think and report of 2-4 different types of sensible stopping criteria by thinking yourself and by consulting the literature/internet. One of them should be to stop when the maximum number of allowed iterations has been reach (to avoid an infinite loop). What would be their pros and cons? Which of these would take into account and "inform" you if your never managed to converge to a minimum? In general, how could you discover non-convergence/divergence?

Exercise B.3
Consider the univariate function

$$f(x) = -e^{-(x-3.14)^2} \tag{7}$$

(a) Write down pseudo-code corresponding to Algorithm 3, but replace the general function $f(x)$ by the actual function of the current problem and its gradient.

(b) Plot the function within an interval $[0, 6]$. Where is the minimum of the function within the range? Could you have figured that out just by looking at the function?

(c) Code up the Gradient Decent algorithm from your pseudo-code in actual Python [you should use NumPy expressions] using a suitable starting guess.

(d) Report on the location and value of the minimum in plot-form and also show in this plot the trajectory of all the values x_t in each iteration, along the way. You can do this simply by creating two empty lists which you call, say `trajectory_x` and `trajectory_fx`. Then you append all x_t to `trajectory_x` and all corresponding $f(x_t)$ to `trajectory_fx`. Thus do something like the following:

```
trajectory_x = []
trajectory_fx = []

# Your Gradient Decent Code
for i in range(N_GD_iterations):
    # ...
    # ... Your GD code
    # ...
    trajectory_x.append(x_t)
    trajectory_fx.append(f(x))

plt.plot(trajectory_x, trajectory_fx)
```

Note: You should plot the trajectory on top of the actual graph for the function from subproblem (b).

(e) The above problem concerns minimization. If the problem would have been instead to consider the function

$$f(x) = e^{-(x-3.14)^2} \tag{8}$$

and were asked to find the maximum locating and value: How would you solve this and what part of the Gradient Descent algorithm would you modify?

4 Task C: Auto-differentiation Using PyTorch

Let us step back for a second. If we compare the workflow and problem solving mechanics of "Grandma's and Grandpa's" style of optimization with Gradient Decent, we can see and hopefully appreciate that when we do Gradient Decent, we do not have the burden of solving a system of equations of the form $\nabla f(\mathbf{x}) = \mathbf{0}$. However, we still have to manually calculate expressions of the derivative/gradient of our objective function and define/implement them in Python code. For simple problems this okay, but as the numbers of variables increase and/or if the functions become increasingly complex and intricate, the thought of having to compute symbolic expressions for derivatives/gradients drowns the excitement of trying to solving optimization problems. There are essentially three alternative ways out from this situation:

- *Symbolic differentiation:* Employ symbolic differentiation using dedicated engines like Mathematica and Maple perform the required symbolic differentiation to get the analytical expressions for the partial derivatives of interest. Such engines takes a function like $f(x, y) = xy + y$ as an input and returns for example the symbolic expression $x + 1$ when the user asks it to return $\frac{\partial f}{\partial y}$. Unfortunately this is not also the most promising road forward as it typically results in very complex expression that are tedious and error prone to implement as Python code. Moreover, a straightforward implementation of the analytical expression without exploiting recurrent substructures within the, will not result in efficient code/computations. Note: There are also tools for symbolic differentiation available in Matlab, Python etc, which partly reduces the problem of implementing the resulting complex expressions.
- *Finite difference approximation:* Determine an estimate of the gradient of a function by performing so called finite difference approximations. The idea here is that the partial derivatives like $\frac{\partial f}{\partial y}$ of a function $f(x, y)$ can be approximated well as

  $$\frac{\partial f}{\partial y} \approx \frac{f(x, y + \delta) - f(x, y)}{\delta}$$

  when δ is chosen to be a very small value. This is a simple and transparent solution but one major limitation is that such calculations become slow when the number of variables increases. Another major limitation is that the numerical values obtained will come with limited precision due to round-off errors.
- *Auto-differentiation:* Employ auto-differentiation, a methodology that exploits the fact that every computer program, no matter how complicated, executes a sequence of elementary arithmetic operations (addition, subtraction, multiplication, division, etc.) and elementary functions (exp, log, sin, cos, etc.) under the hood. By applying the chain rule repeatedly to these operations, and exploiting the fact that the analytical expressions for the derivatives of these elementary functions are known exactly, partial derivatives of arbitrary order can be computed automatically. For some more details regarding the general idea, see the appendix. Although the original ideas are from the 1960s, it is only relatively recently that Auto-differentiation has become popular, especially in the context of training Artificial Neural Networks where this methodology has been a game changer.

  For example, if the computer has implemented the function $f(x_1, x_2) = \ln(x_1 x_2) + x_1 x_2 - sin(x_2)$ as a computational graph, then one can determine any of the partial derivatives $\frac{\partial f}{\partial x_i}$ exactly. This can be achieved by simply employing the well known laws/rules of differentiation of elementary functions, in combination with the rule for derivation of products, and the chain rule. For example $\frac{\ln x_1 x_2}{\partial x_1} = x_2/x_1$. Moreover, by means of clever reuse of previously calculated partial derivatives , other partial derivatives can be determined quicker compared to calculating them from scratch. This idea to reuse results was reinvented in the context of optimizing ANNs already in the 1970s and 1980s, and the corresponding algorithm is known as the **backpropagation algorithm**.

In the following you will learn how to do Auto-differentiation using the Python library; ***PyTorch***. PyTorch was developed by Facebook (now Meta) with building and training deep Artificial Neural

Networks in mind. In PyTorch, this engine is called *autograd*, and is something we need to to import in any notebook or script where we want to use it. In addition to PyTorch, there are several computational frameworks available for Auto-differentiation. Among them, Tensorflow2 (developed by Google) is together with PyTorch currently the most popular because both of them have been designed specifically with training of Artificial Neural Networks in mind.

4.1 Literature and instructions to read before starting

Notably, all code here assumes that the Python version used is **Python> 3.6**. When you are running your code in Google Colab, this is already taken care of.

4.1.1 Mandatory - Check PyTorch Introduction Jupyter Notebook

In order for your to benefit most from this section and be able to more easily solve the problems, it is highly crucial that you have looked at, run and understood the accompanying file:

PyTorch Introduction Jupyter Notebook

which is available on the course home page. There you will learn about how to operate with the basic datatype of PyTorch, namely the ***tensor***. So you must ***do this now - before you read any further***.

4.1.2 Converting to and from PyTorch Tensors

The PyTorch framework is based on a datatype called tensor, which you to a large extent can think of as a numerical array used in NumPy. Here follows some basic examples how to convert to and from this datatype.

```
#from PyTorch tensor to NumPy array
a_as_pytorch_tensor=torch.ones(5)
a_as_numpy_array=a_as_pytorch_tensor.numpy()

#from PyTorch tensor to list or single number
a_as_2D_pytoch_tensor=torch.randn(2,2)
a_as_list=a_as_2D_pytorch_tensor.tolist()
a_single_number=a_as_2D_pytorch_tenstor[0,0].tolist()

#from numpy array to PyTorch tensor
b_as_numpy-array=numpy.ones(5)
b_as_pytoch_tensor=torch.from_numpy(b_as_numpy_array)

#from list to PyTorch tensor
a_as_list = [1.5, 2.9, 3.1]
a_as_pytorch_tensor = torch.FloatTensor(a_as_list)
```

However, be careful to check that the conversion you wanted actually was performed, sometimes you do not get back what you expected. For example, when it comes to conversion from a tensor to a NumPy array, it is not always enough to use `a_as_pytorch_tensor.numpy()` as suggested above. As show in the following code, this conversion only works if the tensor setting is put to the default value `requires_gradient=False`. From the code example below, the conclusion is that one should always use the more general expression `a_as_pytorch_tensor.detach().numpy()`, which works for both cases.

```
# import torch library
import torch
import numpy
```

```
4
5  print('# define a tensor x1 with requires requires_grad = True')
6  x1 = torch.tensor([1.,2.], requires_grad = True)
7  print("x1:", x1)
8
9  print("\n")
10 print('# define a tensor x2 with requires_grad = False')
11 x2 = torch.tensor([1.,2.], requires_grad = False)
12 print("x2:", x2)
13
14 print("\n")
15 print("# Check - default setting is requires_grad = False")
16 x3 = torch.tensor([1.,2.])
17 print("x3:", x3)
18
19 print("\n")
20 print('# x1.numpy() would give an error, need to "detach" first')
21 print("# Expected error message: Can't call numpy() on Tensor that requires grad. Use
       tensor.detach().numpy() instead.")
22 print("x1_np = x1.detach().numpy()")
23 x1_np = x1.detach().numpy()
24 print("x1_np:", x1_np)
25 print("type of x1_np:", type(x1_np))
26
27 print("\n")
28 print('# However x2.numpy() works because here requires_grad = False')
29 print("x2_np =x2.numpy()")
30 x2_np=x2.numpy()
31 print("x2_np:", x2_np)
32 print("type of x2_np:", type(x2_np))
33
34 print("\n")
35 print('"detach" can be applied without any effect to any tensor with requires_grad =
      False')
36 print("x2_np = x2.detach().numpy()")
37 x2_np_detach=x2.detach().numpy()
38 print("x2_np_detach:", x2_np_detach)
39 print("type of x2_np_detach:", type(x2_np_detach))
40 print("CONCLUSION: Always use tensor.detach().numpy() regardless if requires_grad=
      False or not")
41
42 print("\n")
43 print('From the numpy we can create a tensor again')
44 print("x1_tch=torch.tensor(x1_np)")
45 x1_tch=torch.tensor(x1_np)
46 print("x1_tch:", x1_tch)
```

4.1.3 Determining gradients using PyTorch tensors

Like NumPy, PyTorch also forces you to deal with its own type of "data-container" type that represents multidimensional arrays (scalars, vectors, matrices etc). Recall that in NumPy we needed to use ndarrays. The story is the same for PyTorch as well, where the ndarray equivalent in PyTorch is called tensor. One of the great things with PyTorch and the other frameworks available for auto-differentiation is that it lets you define mathematical functions like we normally would in Python via the pattern:

```
1      def f(x):
2          # math expressions ...
3          return function_value_at_x
```

PyTorch makes it very intuitive to determine partial derivatives of the above function using its auto-differentiation engine ***autograd.*** In the next couple of examples, we'll go through how to do this for various illustrative cases.

4.1.4 Scalars and some basic operators need NOT to be tensors

After the imports `import torch` as well as `from torch import autograd`, PyTorch will automatically recognise the most common mathematical binary operations like '+' and '**' and '*' and convert these internally to their equivalent valid torch operations. This means that we don't have to strictly write out these most basic operations using `torch` math operations. Similarly, PyTorch is also smart enough to deal with for example multiplication and addition of scalars without needing to explicitly define these as tensors as well.

4.2 Examples and Exercices

4.2.1 Example 1: PyTorch auto-differentiation - univariate function

The function $f(x) = 3x^2 + 1$ has the derivative $\frac{df}{dx}(x) = 6x$. We wish to evaluate the derivative of the function at $x = 2$. If we do this with any framework, we would expect to get a value of $\frac{df}{dx}(x = 2) = 12$. So lets see how we do this the PyTorch way:

1. First define the function as any normal Python function. Normally you might have done this using NumPy operations, but here you must instead exclusively use `torch.tensors` and torch operations and other valid PyTorch operations:

```
    import torch
    from torch import autograd

    def f(x):
        return 3*torch.pow(x,2) + 1

```

2. Second, create a tensor with the value `x0` specifying where we wish to evaluate the function and its derivative. Assume we want to evaluate at $x_0 = 2.0$. This is done by setting the `requires_grad` argument to `True`:

```
    x0 = torch.tensor(2.0, requires_grad=True)

```

3. Next we evaluate the function at the specific `x0` and then perform the key `.backward()` operation that calls the autograd engine and makes auto-differentiation happen:

```
    fx0 = f(x0)
    fx0.backward()

```

4. The value of the derivative $\frac{df}{dx}(2.0)$ is now found in the `.grad` attribute of the `x0` tensor:

```
    x0.grad # print this is to see that its a tensor with a value of 12.0

```

This is all there is to it: Assume we wanted to also compute the derivative of the above function in a couple of other locations, say $x_1 = 1.0$ and $x_3 = 5.0$. Now we expect to get $\frac{df}{dx}(1.0) = 6$ and $\frac{df}{dx}(5.0) = 30$, respectively. We achieve this by simply doing as above for each new evaluation position of interest:

```
x1 = torch.tensor([1.], requires_grad=True)
x2 = torch.tensor([5.], requires_grad=True)

fx1 = f(x1)
fx2 = f(x2)

fx1.backward()
fx2.backward()

x1.grad # print these and check that you get back a 6.0 tensor
x2.grad # print these and check that you get back a 30.0 tensor
```

4.2.2 Exercise C.1

Consider the function

$$f(x) = (x-1)^4 + 3 \tag{9}$$

1. Compute the symbolic derivative of the above function by hand. Then compute the value of derivative of the function at $x = 2$, i.e $\frac{df}{dx}(x = 2)$.
2. Code up the above function as a Python function using torch mathematical expressions (otherwise autograd will not work). Use PyTorch auto-differentation to get the same result of the derivative (gradient) at $x = 2$.
3. Confirm that the value is the same as the one you calculated by hand.

4.2.3 Exercise C.2

(a) Make sure you understand that the following code is implementing the function $f(z) = 1+z+z^2$ using a for loop and PyTorch commands.

```
import torch
from torch import autograd
import numpy as np

#Implements the function f(z)=1+z+z^2 using a for loop
def f_loop(z):
    start=0
    stop_plus_one=3
    step=1
    n_values=np.arange(start,stop_plus_one,step)

    s=0
    for n in n_values:
      s=s+torch.pow(z,n)
      n=n+1

    return s
```

In particular, run the following code snippet to make sure the function returns what you expect for different values of z like $z \in \{1.0, 2.0, 3.0\}$ by doing the required calculations by hand.

```
z0=torch.tensor(2.0,requires_grad=True)
fz0=f_loop(z0)
fz0
```

Finally, run the following code snippet to calculate the derivative. Make sure the function returns what you expect for different values of z like $z \in \{1.0, 2.0, 3.0\}$ by doing the required calculations by hand.

```
fz0.backward()
z0.grad
```

Remark
This last exercise shows that PyTorch is very generally applicable: The mathematical function $f(z) = 1+z+z^2$ is implemented by means of a for loop in this case, meaning not using a conventional mathematical expressions. Still PyTorch can automatically calculate the corresponding gradients! You can try to repeat the above steps using the perhaps more conventional implementation shown here below:

```
def f_no_loop(z):
    s=1+z+torch.pow(z,2)
    return s
```

```
z1=torch.tensor(2.0,requires_grad=True)
fz1=f_no_loop(z1)
fz1
```

```
fz1.backward()
z1.grad
```

4.2.4 Example 2: PyTorch auto-differentiation - multivariable functions

In this example we show how to perform auto-differentiation of functions of several variables with PyTorch. The function we care about is $g(w, y, z) = x^2 + 2y^2 + wyz$. It's three partial derivatives are

$$\frac{\partial g}{\partial w}(w, y, z) = 2w + yz \tag{10}$$

$$\frac{\partial g}{\partial y}(w, y, z) = 4y + wz \tag{11}$$

$$\frac{\partial g}{\partial z}(w, y, z) = 2w + wy \tag{12}$$

We want to evaluate g at the point $(w = 1.0, y = 2.0, z = 3.0)$, and we also want to determine the values of all three partial derivatives at that point as well. We do this very much like in the univariate case:

1. We define the function of interest as a Python function.

```
    def g(w,y,z):
        return w**2 + 2*(y**2) + w*y*z

```

2. We create the PyTorch tensors with the values we want, in order to evaluate the function and all its partial derivatives at the point of interest $(w0 = 1.0, y0 = 2.0, z0 = 3.0)$:

```
    w0 = torch.tensor(1.0, requires_grad=True)
    y0 = torch.tensor(2.0, requires_grad=True)
    z0 = torch.tensor(3.0, requires_grad=True)

```

3. Evaluate function at the point of interest $(w0 = 1.0, y0 = 2.0, z0 = 3.0)$:

```
    g_value = g(w0, y0, z0) # print to check = 15.0

```

4. call `.backward()` on function result to perform auto-differentiation of all three partial derivatives.

```
    g_value.backward()

```

5. The values of $\frac{\partial g}{\partial w}(w_0, y_0, z_0)$, $\frac{\partial g}{\partial y}(w_0, y_0, z_0)$ and $\frac{\partial g}{\partial z}(w_0, y_0, z_0)$ can be accessed as follows:

```

    w0.grad # = dfdw(w0, y0, z0), should produce tensor(8.)
    y0.grad # = dfdy(w0, y0, z0), should produce  tensor(11.)
    z0.grad # = dfdz(w0, y0, z0), should produce tensor(2.)


```

4.2.5 Excercise C.3

Consider the function

$$f(x, y, z) = xy + xz + yz + x^2 + e^y + zsin(x) \tag{13}$$

1. Compute the expression for $\nabla f(x, y, z)$ by hand. In other words, compute the symbolic expressions for the gradient of the function f, containing all 3 three partial derivatives. Show the expressions for each partial derivative of f.
2. What is the gradient of f at the point $(x_0 = \pi, y_0 = 2.0 z_0 = 2.0)$?
3. Determine the gradient of f using auto-differentiation in PyTorch.
4. Confirm that the value is the same as the one you calculated by hand and then by means of a hand calculator or a small python code.

Tip: Here are the correct values:

```
dfdx_true= 8.283180000007041
dfdy_true= 12.53064609893065
dfdz_true= 5.141592653589793
```

4.3 Exercise C.4: PyTorch auto-differentiation - functions of a vector

The task here is to implement the following example, and reflect on the results obtained and the associated theory. Consider the function $f(\mathbf{x})$ where $\mathbf{x} \in \mathbb{R}^4$, with a vector value input $\mathbf{x} = [x_1, x_2, x_3, x_4]^\top$, defined by

$$\begin{aligned} f(\mathbf{x}) = f([x_1, x_2, x_3, x_4]^\top) &= (\mathbf{x} - \mathbf{1})^\top (\mathbf{x} - \mathbf{1}) \\ &= (x_1 - 1)^2 + (x_2 - 1)^2 + (x_3 - 1)^2 + (x_4 - 1)^2 \end{aligned}$$

where $\mathbf{1} = [1.0, 1.0, 1.0, 1.0]^\top$ is nothing more than vector of ones used to achieve a convenient compact notation. The gradient of the function $f(\mathbf{x})$ is the column vector

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}(\mathbf{x}), \frac{\partial f}{\partial x_3}(\mathbf{x}), \frac{\partial f}{\partial x_3}(\mathbf{x}), \frac{\partial f}{\partial x_4}(\mathbf{x}) \right]^\top$$

where $\frac{\partial f}{\partial x_i}(\mathbf{x}) = 2(x_i - 1)$, $i = 1, 2, 3, 4$.

Now make sure you fully understand what a gradient is before continuing this example. If the above is not obvious to you or you feel a bit sketchy about what the gradients means: Please use this as a warning sign to urge you to refresh your mental model of what a gradient of a scalar valued function is, both mathematically and intuitively, and how to practically compute gradients by hand. The internet is your friend.

Assume we are interested in the function values and the gradients at a several points, lets say these are $\mathbf{x}_0 = [0., 0., 0., 0.]$, $\mathbf{x}_1 = [1., 1., 1., 1.]$, and $\mathbf{x}_2 = [1.5, 1.5, 1.5, 1.5]$. The procedure to get these with PyTorch is *almost* identical to the previous examples. The only difference here is that the input to the function is a vector (stored as a 1D PyTorch tensor) of length 4, rather than several individual scalar variables.

Before you read further about what PyTorch outputs, it is recommended that you invest 5 minutes to calculate the function value and the gradient of the function at the three points $\mathbf{x}_0$, $\mathbf{x}_1$, $\mathbf{x}_2$ of interest by hand.

1. Define the function as a Python function using valid PyTorch math operations

```
1
2     def f(x):
3         one_vector = torch.ones_like(x)
4         return (x - one_vector).T @ (x - one_vector)
5
```

2. Create a tensor with values corresponding to the the points of interest. We'll show you the case for $\mathbf{x_0}$ and you should yourself redo the steps for $\mathbf{x_1}$ and $\mathbf{x_2}$:

```
1
2     x0 = torch.zeros(size=[4], requires_grad=True)
3
```

   which should look like `tensor([0., 0., 0., 0.], requires_grad=True)` when printed.

3. Evaluate the function value at the point of interest. We should expect the result is a tensor with value 4.0

```
1
2         fx0 = f(x0)
3
```

4. Now for the gradient computation: We simply apply `.backward()` on the result `fx0`:

```
1
2       fx0.backward()
3
```

The tensor representing $\nabla f(\mathbf{x}_0)$ can now be accessed using *.grad*:

```
1
2   x0.grad # print and check  = "tensor([-2., -2., -2., -2.])"
3
```

Remark
One should note that not all PyTorch tensors need to have the specification `requires_grad=True`. One example of this is the constant tensor `one_vector` above, consisting only of the value one in each position. This is because we will never want to differentiate with respect to such a constant vector or variable.

5 Task D: Gradient Descent the PyTorch Way

All examples above have hopefully given you the general rhythm and melody of how to perform auto-differentiation with PyTorch. This will be another tool in your mental and technical tool-box to use when faced with either a differentiation problem or an optimization problem.

5.1 Code Template - Gradient Descent the PyTorch Way

When in comes to performing gradient descent based optimization, since the examples above demonstrate how to automate the task of computing the gradient $\nabla f(\mathbf{x}_t)$ of a function f at a specific point $\mathbf{x}_k$), the hardest part of the gradient decent algorithm update $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta_t \nabla f(\mathbf{x}_t)$ is now solved for us. The general way to perform Gradient Decent using PyTorch is shown below as a "Code-template". Notably, inside the code there are some technical notes, which explains some particular features of the PyTorch implementation.

However **you only need to think about these three PyTorch specific technical details when you want to implement Gradient Decent and similar algorithms yourself**. If you instead want to use standard algorithms such as plain Gradient Descent or improved/stochastic versions of it like the one called Adam, there is already a so called PyTorch `optimizer` object, which takes care of these technical details.

```

####### Code template: The Gradient Decentalgorithm in PyTorch #########
import torch
from torch import autograd # !!

# Define you objective function
def f(x, y, ....):
    # ...
    # math expression using only valid torch operations
    # ...
    return output_value

x = torch.tensor(...., requires_grad=True) # initial starting guess

# ...

eta = # multiplication factor that determines the actual step
      # length  eta||g||, where g is the gradient vector

for i in range(nr_steps):
    # compute function at x
    fx = f(x)

    # compute gradient -> 'populate' all partial derivatives
    fx.backward()

    # gradient decent update.
    #************************************************
    #***NOTE SEE TECHNICAL NOTE AT END OF THIS SCRIPT***
    #************************************************
    with torch.no_grad():
        x -= eta * x.grad # needs to be written like this
        # ...

    x.grad.zero_()

    if some_stopping_criteria_fulfilled:
        break


    #********************************************************
```

```
42    #                        TECHNICAL NOTES:
43    #*********************************************************
44    # Gradient Descent: THREE PYTORCH SPECIFIC TECHNICALITIES TO HANDLE
45    #
46    #       1. One technicality is that the perhaps intuitive code
47    #       snippet x = x - eta * x.grad will not work!
48    #       This is because PyTorch looses the intermediate result obtained in the
49    #       right side before it is assigned to the left side. This means
50    #       one has to use the "right-in place assignment"  x -= eta * x.grad
51    #       that does not create any intermediate copies but directly updates x
52    #
53    #       2. Another technicality here is PyTorch's default behaviour to build a
54    #       dynamic computational graph from every Python operation that involves
55    #       any gradient-requesting  tensor or its dependencies. However
56    #       torch.no_grad() allows us to  perform regular Python operations on
57    #       tensors, without  performing gradient calculations. Therefore the update
58    #                          x -= eta * x.grad has
59    #       must take place inside the scope of torch.no_grad()
60    #
61    #       3. A third odd technicality here is that the default behaviour of
62    #       PyTorch is to accumulate gradients across iterations. Thus one gradient
63    #       calculation will be aded on top of the previous gradient calculation.
64    #       Therefore the gradient field in the  tensor must be reset to zero in
65    #       each iteration. This is achieved  using x.grad.zero_()
```

In addition to the technical notes regarding how to use the PyTorch implementation, which provided inside the code template, here follows the same information written out in a better typographic format:

1. The update equation must be implemented using the "right in-place assignment" format `x -= step_length * x.grad`, meaning that the semantically equivalent
   `x = x- step_length * x.grad`
   is not working. This is because the second alternative will result in an intermediate calculation and storage of the right hand side components, before the memory location of `x` is overwritten with the result of the computations performed. By contrast the "in-place assignment" means that the memory location of `x` will be overwritten directly. Due to how PyTorch was implemented, the gradient part becomes lost if the second alternative is used, whereas the "in-place assignment" format `x -= step_length * x.grad` works.

2. You must do the Gradient Descent step using `with torch.no_grad():`, see line 31 in the Gradient Descent code-template above. As briefly explained in the code comments, `torch.no_grad()` allows us to perform regular Python operations on tensors, not involving gradient calculations using a redefined computational graph. This is exactly what we want here - because we want to update the part of the tensor `x` storing the current position, without changing the already built computational graph. Another view/explanation is that without employing `torch.no_grad()`, the update equation will on the right side involve subtracting two tensors with different properties: the full tensor `x` and the gradient tensor `x.grad`. Thus by activating `torch.no_grad()`, the gradient part of `x` is becoming "invisible", thereby making it compatible with the tensor `torch.no_grad()` in the update equation.

3. After each Gradient Descent update, the code must perform a "set gradient back to zero" operation. This removes the gradient computed before, which is stored in `x.grad`. If you read the section below, called "Mental Model of PyTorch Tensors", you'll find that if the gradient is not "reset" back to zero, then in practice PyTorch will add to the current sum of gradients the new gradients obtained in the current iteration. That's why the code needs to reset the gradient by making the call `x.grad.zero_()`, see line 35 in the Gradient Descent code-template above.

5.2 Exercises - Gradient Descent the PyTorch Way

Exercise D.1: PyTorch Gradient Descent - one variable

$$f(x) = -e^{-(x-\pi)^2} + 0.01 * (x-\pi)^2 \tag{14}$$

1. Plot the above function on the interval $[-5, 11]$.
2. Find the minimum of the above function by coding up the problem and solution in PyTorch using 1000 steps. Try different step lengths, observe the behaviour a step size $\eta = 1.0$ and when $\eta = 0.1$, and $\eta = 0.01$, Also track the trajectory of your solution and overlay them on top of the plot- NOTE: In order to plot to curves on top of each other, both plotting commands must be located inside the same cell.

Tip: Here is a code snippet that can be of value when you are going to collect data for the plotting. One issue here is how to convert from PyTorch tensor format to floats suitable for plotting, and this is what the following code can help you with.

```
  fx=f(x)

  x_np=x.detach().numpy()
  x_np=float(x_np)
  trajectory_x.append(x_np)

  f_value_np=fx.detach().numpy()
  trajectory_y.append(f_value_np)
```

Exercise D.2: PyTorch Gradient Descent - two variables

The following function

$$f(x_1, x_2) = 4(x_1 - 3)^2 + (x_2 - 2)^2$$

has a global minimum at the point

$$[x_1 = 3.0, x_2 = 2.0].$$

Confirm that this is the case by coding up a Gradient Decent minimization procedure, starting at the guess $[x_1 = -7.0, x_2 = -8.0]$. Track the solution history and plot these solutions for each iteration on top of a contour plot of the above function. Use a suitable step size that doesn't result in 'zig-zag' trajectory towards and around the minimum. Try the following values of the step parameter η: $\{0.01, 0.03, 0.1, 1\}$ What was the value of the step length that did the job best?

Tip:
Here comes some code snippets that can be very helpful for this task. First, the function of interest $f(x_1, x_2) = 4(x_1 - 3)^2 + (x_2 - 2)^2$ is implemented in two versions. The first one, `f_float`, is for producing the values needed when employing `plt.contour`. The second function, `f_torch`, is for performing the PyTorch based Gradient Descent.

```
def f_float(x1,x2):
  return (2*(x1-3))**2+(x2-2)**2

import torch
def f_torch(x):
  x1=x[0]
  x2=x[1]
  return torch.pow(2*(x1-3),2)+torch.pow(x2-2,2)
```

The following code gives the start guess for the Gradient Descent as a PyTorch tensor.

```
x_start=np.asarray([-7.0, -8.0])
x_start
x=torch.tensor(x_start,requires_grad=True);
x
```

The following is to perform the actual Gradient Descent, and in each step store the current location in the two lists `trajectory_x1` and `trajectory_x2`:

```
n_max=100
trajectory_x1=[]
trajectory_x2=[]
trajectory_y=[]
eta=0.03
for n in range(n_max):

  fx=f_torch(x)

  x_np=x.detach().numpy()
  trajectory_x1.append(x_np[0])
  trajectory_x2.append(x_np[1])

  f_value_np=fx.detach().numpy()
  trajectory_y.append(f_value_np)

  fx.backward()

  with torch.no_grad():
    x-=eta*x.grad

  x.grad.zero_()
```

Finally, the desired contour plot and the Gradient Descent trajectory are plotted. Shown as a blue dot is also the desired optimal point.

```
# Define a grid of x and y values
x = np.linspace(-12, 12, 100)
y = np.linspace(-10, 10, 100)

Z=np.empty([100,100])
for i in range(100):
  for j in range(100):
    Z[j,i]=f_float(x[i],y[j]) #NOTE: Indices [j,i] are switched to make
                              #      it compatible with plt.contour


# Create the contour plot
contour_plot = plt.contour(x, y, Z, levels=15, cmap='viridis')
plt.colorbar(contour_plot, label='Function Value')
plt.xlabel('x1')
plt.ylabel('x2')
plt.title('Contour Plot')

# Overlay the trajectory on the contour plot
plt.plot(trajectory_x1, trajectory_x2, marker='.', linestyle='-', color='red', label=
    'Trajectory')
plt.plot(3,2, marker='.', linestyle='-', color='blue',label='Optimum')
plt.show()
```

An example result is shown in Fig.3.

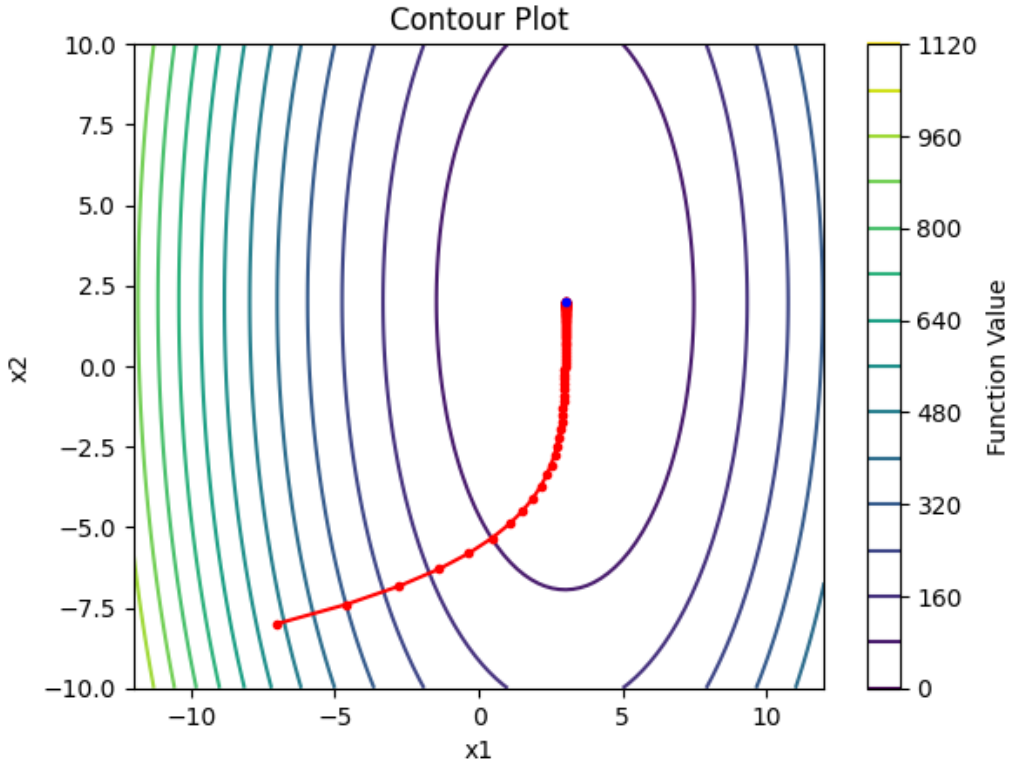


Figure 3: Gradient Descent trajectory example, starting at the guess $[x_1 = -7.0, x_2 = -8.0]$.

Exercise D.3: The Unavoidable Problem of Local Minima - one dimension
Consider the function

$$f(x) = -e^{-(x-1)^2} - 5e^{-(x-10)^2}$$

(a) Plot the above function in the interval $[-10, 20]$. How many minima does this function have? Explain why you think Gradient Descent might have problems finding the global minimum in this case.
(b) Implement again the usual Gradient Decent algorithm using PyTorch as you have done before in order to find the minima of a function. Remember that your function has to be implemented using torch functions (in order for the auto-differentiation to work)!
(c) Based on the plot of the function made before, where would you put a good initial starting position $\mathbf{x}_0$? Why is this a good choice?
(d) Run the Gradient Descent 1000 steps twices, first when starting from the location `x0=7.0` and then when starting from `x0=3.0`. Use as step length parameter value `eta=0.1`. Do you find that the algorithm ends up in the global minimum in both cases, or not?
(e) Run the Gradient Descent again starting at `x0=7.0`, but now using `eta=0.01` and `eta=1.0`. Did the Gradient Descent work successfully in any/both cases?

Tip: To plot the points for the trajectory, remember that both the plot of the function and the trajectory must take place in the same cell. Moreover, this following code snippet can be useful. Here follows also an example plot in Fig.4.

```
marker_size=12
plt.scatter(trajectory_x,trajectory_y,s=marker_size,color='b')
plt.show()
```

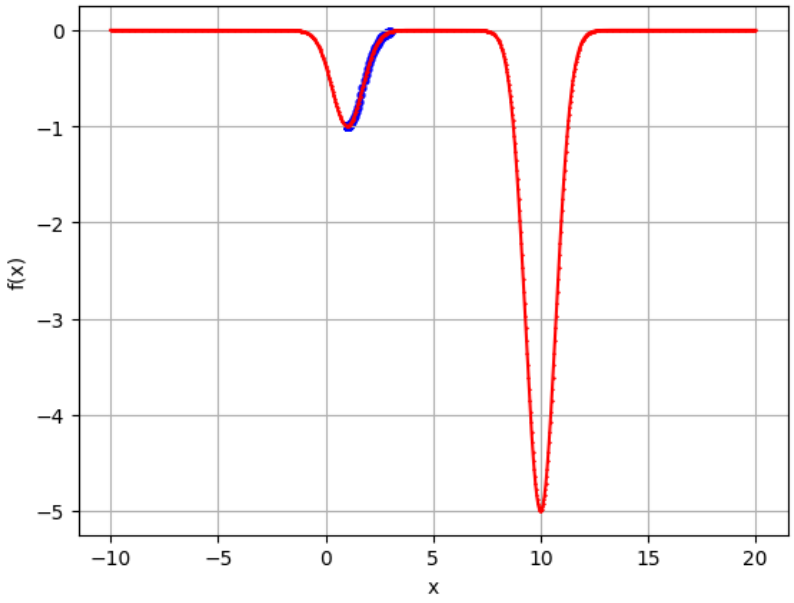


Figure 4: Gradient Descent trajectory example, ending up in a local minimum.

6 Task E: Optimal Can shape for the The Coca Koala Company

Scenario: You've applied for a data-scientist job at The Coca Koala Company, a major beverage company in Australia and you have been fortunate enough to be called in for an interview after their requiters have been impressed with your CV. Coca Koala has exactly zero problems making sweet drinks that get kids and adults hooked for life, but what they do have a major problem with is the shape of their aluminium cans. They are spending way too much money on aluminium for their aluminum cans, steaming from the fact the company believes that they have sub-optimal dimensions for the cylindrical can. Coca Koala buys aluminium in bulk and desires a can shape that can contain the same exact amount of volume of their current cans (volume = 355 cl = 355 $(\text{cm})^3$) but with as little aluminium used as possible.

At your interview, you are tasked to formulate this problem as an optimization problem (to demonstrate modelling skills) and to solve it using Python (to demonstrate both programming skills and ability to fluidly transition from idea to execution). What are your optimal can dimensions in cm and what would be the optimal material use per can? The current cans have a surface area of 323 $(\text{cm})^2$. How much less material would your solution use? Thus your tasks are as follows:

(a) Formulate the above problem as a simple optimization problem in one line of text using words like, objective function and type of optimization problem and the algorithm approach to solving the optimization problem.

(b) Make a clear explanation showing that your mathematical formulation of the problem is reasonable, how you simplify the scenario given, and how you code up and solve the problem using what you have learned so far. Include plots of the objective function of relevance here.

Tip
Recall the formulas for the volume and surface of a cylinder: $V = h \cdot \pi \cdot r^2$ and $A = 2 \cdot \pi \cdot r \cdot h + 2\pi \cdot r^2$. Then use the constraint that the volume must be $V = 355$ to obtain objective function of only one variable.

Remark
You may solve this problem using the PyTorch machinery, or you can use a standard NumPy approach to implement the required Gradient Descent algorithm.

7 General Remarks

7.1 A Useful Mental Model of PyTorch Tensors

You might have noticed something that perhaps looks a bit weird, including why is the gradient vector, `x0.grad` stored inside and as a part of the original tensor `x0`. When we create a PyTorch tensor, we are actually creating a complex object that not only contains the data (tensor element values), but this object has several other fields which can be accessed too.

For example, accessing the shape of a tensor `x` is simply done with $x.shape$, just like you would when accessing the attributes of other python objects. One of these attributes, or *fields*, in the tensor object is the `.grad`, which is initialized as an empty None field. It is initially "empty" as PyTorch hasn not been instructed to figure out which function to determine the gradient for at the particular location `x`.

It is only when you use `x` as an input to some function and call `.backward()` using on the current function value that you will make PyTorch populate the `.grad` field of the `x`. To help your intuition, Fig. 5 shows roughly what your mental model of a tensor object should be, and how it gets modified when performing PyTorch auto-differentiation.

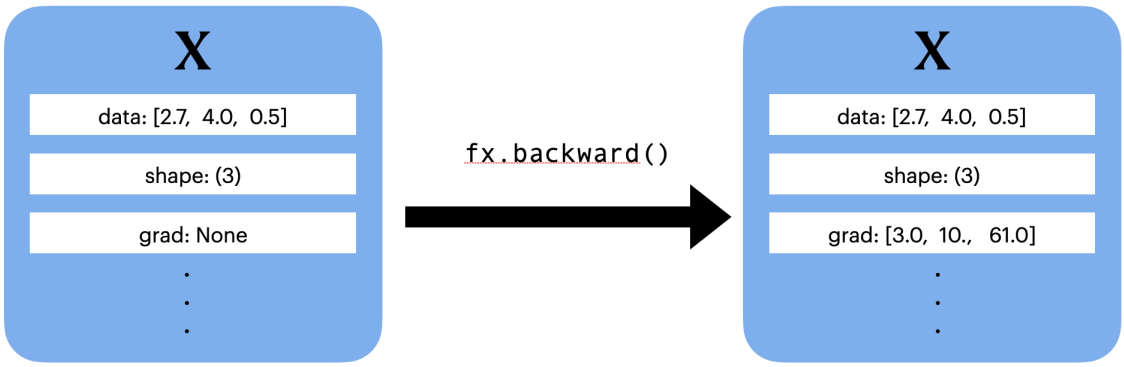


Figure 5: Illustration of a "mental model" of a PyTorch tensor before and after calling `.backward()`, which invokes auto-differentiation.

7.2 Beyond Gradient Descent Based Optimization

Optimization by Gradient Descent and similar methods are powerful to use when the goal is to find a local optimum for objective functions of interest. However, there are three major limitations of Gradient Decent that are important to understand:

- ***Gradients must exist:*** One limitation is that Gradient Decent requires the objective function to actually have a derivative/gradient, i.e. the functions need to be differentiable. If your objective function $f(\mathbf{x})$ is not differentiable, then the gradient $\nabla f(\mathbf{x})$ doesn not make any sense, which in turn means that the "update" step to get $\mathbf{x}_{t+1}$ breaks down.
- ***Often convergence to unwanted local minima:*** Gradient Descent will only find a local minimum, so if the objective function has many local minima that are much

worse than the desired global minimum, then the Gradient Descent search will most likely converge to one of these unwanted minima.

- ***Slow convergence:***Gradient Decent will often need many many iterations/updates to reach a local minimum. This in turn means that we will have to evaluate the objective function $f(\mathbf{x})$ and its gradient $\nabla f(\mathbf{x})$ many many times (iterations). This becomes a big problem when the evaluation of $f(\mathbf{x})$ and/or its gradient $\nabla f(\mathbf{x})$ is "costly". Here "costly" may mean "taking a lot of time" (e.g. performing long practical experiments that involves many manual steps or long computer processing time), or "economically expensive", or both. Therefore, if your time and economy budget only allows for a few function evaluations, it becomes very critical to find a "close to a local minimum" in as few steps as possible. In this case, Gradient Descent search might be too inefficient.

8 DEADLINE AND EXAMINATION

This examination will consists of the following parts:

(1) Answers and Results: You uploading to Studium answers and results to all mandatory problems present above in the form of one single pdf document. **Deadline: See course information.**.

NOTE: If you are working in pairs, each person has to submit their own document, even if it is almost identical. You are of course always responsible for your own submission and must be able to explain all of it during seminars and similar examinations.

(2) Python code as one notebook or as wrapped inside a zip file: The code should be ready to run in Google Colab. If your code consists of more than one notebook (and/or some additional code called from inside the notebook) your instructions how to run the code should be provided as a text file called `readme.txt`. Make sure that all files you are submitting have your first name and surname as part of the file name, for example like:

`Assignment_2_reflections_Mats_Gustafsson.pdf`

NOTE: If you are working in pairs, each person has to submit their own zip-file, even if it is almost identical.

(3) Personal reflections and feedback: An uploaded pdf document to Studium providing personal reflection and feedback. This pdf document should contain reflective answers to the following questions. Alternatively, you may also include the answers to these questions in your submitted Google Colab notebook.

(a) Why is Gradient Descent a reasonable method for finding local minima to a function? In particular, which is the basic idea of this algorithm, how is it moving towards a local mimimum?
(b) Why is auto-differentiation very important/useful in the context of Gradient Descent when the function to be minimized is very complex to evaluate and differentiate?
(c) What is the name of second relatively new computational framework, similar to PyTorch, which also was designed with optimization of ANN parameters in mind? This alternative framework was developed by Google in is currently also very popular.
(d) Do you think you understand the key idea presented in Appendix C below regarding the key idea of auto-differentiation? What is the basic principle the auto-differentiation methodology is relying on?
(e) What did you personally find interesting and/or useful an/or informative with the assignment, and why? If you actually did not find it enlightening, in that case we would like to know about why.
(f) What did you find frustrating and/or difficult, and/or unnecessary, and why?
(g) What would you suggest as an improvement of this assignment for next years students, and why?
(h) Did you spend more than 20 hours to complete this exercise? Why?

NOTE: If you are working in pairs, each person has to send in a personal reflection.

(4) Mandatory Seminar: You discussing/presenting your results during the mandatory seminar S1. You do NOT have to prepare any separate slide presentation, it will be fine to show your Colab notebook if/when you are requested to present something.

Good Luck and Enjoy!

= = = = = = = = = = = = = = = = =

Appendix

A Warm up & Repetition

Click this text: *Maxima, minima, and saddle points*
in order move here:
https://www.khanacademy.org/math/multivariable-calculus/applications-of-multivariable-d%erivatives/optimizing-multi%variable-functions/a/maximums-minimums-and-saddle-points

B Autograd in PyTorch

Click this text: *Understanding graphs and automatic differentiation*
in order move here:
https://blog.paperspace.com/PyTorch-101-understanding-graphs-and-automatic-differentiation/

Click this text: *Implementing Gradient Descent in Python, Part 1: The Forward and Backward Pass*
in order move here:
https://blog.paperspace.com/part-1-generic-python-implementation-of-gradient-descent-for-nn-optimization/

Click this text: *Understanding PyTorch with an Example Step-by-Step Tutorial*
in order move here:
https://towardsdatascience.com/understanding-pytorch-with-an-example-a-step-by-step-tutorial-81fc5f8c4e8e

Click this text: *Computational Graphs in PyTorch and Tensorflow*
in order move here:
https://towardsdatascience.com/computational-graphs-in-pytorch-and-tensorflow-c25cc40bdcd1

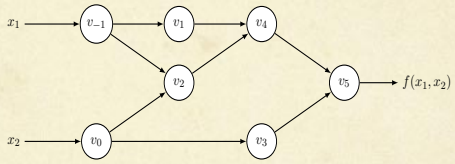
C The general concept of Auto-Differentiation

Auto-Differentiation

The key idea you should understand and remember:

(1) Any function f() implemented in a computer can be represented by a computational graph, with each node consisting only of only one among few elementary functions like +,*, /, x^n, exp(), log(), sin() being hardwired into the computer system.

Example:



Computational graph of the example $f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$.

(2) Exact differentiation with respect to any input variable can therefore be obtained by employing the chain rule and the fact that the derivatives of the elementary functions are known!

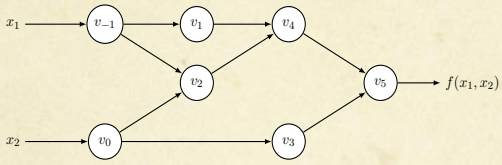
Figure 6: The key idea of Auto-differentiation.

Auto-Differentiation

Basic algorithm:

Given nya function implemented on a computer:

(1) "Unfold" a function call into forward calculations through a computational graph, for example as in the example below:



Computational graph of the example $f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$.

(2) Calculate desired partial derivatives exactly (no numerical errors) by exploting the chain rule and the fact that the derivatives for all operations in the graph are exactly known.

Remark. There are alternative strategies for carrying out these calculations. The "reverse accumulation" (backpropagation) method turns out to be computationally most efficient for the common case when the function has many inputs and produces a scalar (one-dimensional) output.

Figure 7: The basic (backpropagation) algorithm for Auto-differentiation used when there are many input variables.

Auto Differentiation

- Original ideas are from the 1950's and 1960´s, the reverse mode AD (efficient for scalar value functions of many variables) formalized in the 1970s. Many different impelementations have been reported.

Table 5: Survey of AD implementations. Tools developed primarily for machine learning are highlighted in bold.

Language	Tool	Type	Mode	Institution / Project	Reference	URL
AMPL	AMPL	INT	F, R	Bell Laboratories	Fourer et al. (2002)	http://www.ampl.com/
C, C++	ADIC	ST	F, R	Argonne National Laboratory	Bischof et al. (1997)	http://www.mcs.anl.gov/research/projects/adic/
	ADOL-C	OO	F, R	Computational Infrastructure for Operations Research	Walther and Griewank (2012)	https://projects.coin-or.org/ADOL-C
C++	Ceres Solver	LIB	F	Google		http://ceres-solver.org/
	CppAD	OO	F, R	Computational Infrastructure for Operations Research	Bell and Burke (2008)	http://www.coin-or.org/CppAD/
	FADBAD++	OO	F, R	Technical University of Denmark	Bendtsen and Stauning (1996)	http://www.fadbad.com/fadbad.html
	Mxyzptlk	OO	F	Fermi National Accelerator Laboratory	Ostiguy and Michelotti (2007)	
C#	AutoDiff	LIB	R	George Mason Univ., Dept. of Computer Science	Shtof et al. (2013)	http://autodiff.codeplex.com/
F#, C#	**DiffSharp**	OO	F, R	Maynooth University, Microsoft Research Cambridge	Baydin et al. (2016a)	http://diffsharp.github.io
Fortran	ADIFOR	ST	F, R	Argonne National Laboratory	Bischof et al. (1996)	http://www.mcs.anl.gov/research/projects/adifor/
	NAGWare	COM	F, R	Numerical Algorithms Group	Naumann and Riehme (2005)	http://www.nag.co.uk/nagware/Research/ad_overview.asp
	TAMC	ST	R	Max Planck Institute for Meteorology	Giering and Kaminski (1998)	http://autodiff.com/tamc/
Fortran, C	COSY	INT	F	Michigan State Univ., Biomedical and Physical Sci.	Berz et al. (1996)	http://www.bt.pa.msu.edu/index_cosy.htm
	Tapenade	ST	F, R	INRIA Sophia-Antipolis	Hascoët and Pascual (2013)	http://www-sop.inria.fr/tropics/tapenade.html
Haskell	ad	OO	F, R	Haskell package		http://hackage.haskell.org/package/ad
Java	ADiJaC	ST	F, R	University Politehnica of Bucharest	Slusanschi and Dumitrel (2016)	http://adijac.cs.pub.ro
	Deriva	LIB	R	Java & Clojure library		https://github.com/lambder/Deriva
Julia	JuliaDiff	OO	F, R	Julia packages	Revels et al. (2016a)	http://www.juliadiff.org/
Lua	**torch-autograd**	OO	R	Twitter Cortex		https://github.com/twitter/torch-autograd
MATLAB	ADiMat	ST	F, R	Technical University of Darmstadt, Scientific Comp.	Willkomm and Vehreschild (2013)	http://adimat.sc.informatik.tu-darmstadt.de/
	INTLab	OO	F	Hamburg Univ. of Technology, Inst. for Reliable Comp.	Rump (1999)	http://www.ti3.tu-harburg.de/rump/intlab/
	TOMLAB/MAD	OO	F	Cranfield University & Tomlab Optimization Inc.	Forth (2006)	http://tomlab.biz/products/mad
Python	ad	OO	R	Python package		https://pypi.python.org/pypi/ad
	autograd	OO	F, R	Harvard Intelligent Probabilistic Systems Group	Maclaurin (2016)	https://github.com/HIPS/autograd
	Chainer	OO	R	Preferred Networks	Tokui et al. (2015)	https://chainer.org/
	PyTorch	OO	R	PyTorch core team	Paszke et al. (2017)	http://pytorch.org/
	Tangent	ST	F, R	Google Brain	van Merriënboer et al. (2017)	https://github.com/google/tangent
Scheme	R6RS-AD	OO	F, R	Purdue Univ., School of Electrical and Computer Eng.		https://github.com/qobi/R6RS-AD
	Scmutils	OO	F	MIT Computer Science and Artificial Intelligence Lab.	Sussman and Wisdom (2001)	http://groups.csail.mit.edu/mac/users/gjs/6946/refman.txt
	Stalingrad	COM	F, R	Purdue Univ., School of Electrical and Computer Eng.	Pearlmutter and Siskind (2008)	http://www.bcl.hamilton.ie/~qobi/stalingrad/

F: Forward, R: Reverse; COM: Compiler, INT: Interpreter, LIB: Library, OO: Operator overloading, ST: Source transformation

Baydin, Pearlmutter, Radul, and Siskind

- Two recent popular AD frameworks designed especially for training Artificial Neural Networks for use in Python are:
  - **Tensorflow** by Google (more recent development is Tangent)
  - **PyTorch** by Facebook

Figure 8: There are many frameworks offering Auto-Differentiation.

Auto-Differentiation - Wikipedia

The chain rule, forward and reverse accumulation

Fundamental to AD is the decomposition of differentials provided by the chain rule. For the simple composition

$$y = f(g(h(x))) = f(g(h(w_0))) = f(g(w_1)) = f(w_2) = w_3$$
$$w_0 = x$$
$$w_1 = h(w_0)$$
$$w_2 = g(w_1)$$
$$w_3 = f(w_2) = y$$

the chain rule gives

$$\frac{dy}{dx} = \frac{dy}{dw_2}\frac{dw_2}{dw_1}\frac{dw_1}{dx} = \frac{df(w_2)}{dw_2}\frac{dg(w_1)}{dw_1}\frac{dh(w_0)}{dx}$$

Usually, two distinct modes of AD are presented, **forward accumulation** (or **forward mode**) and **reverse accumulation** (or **reverse mode**). Forward accumulation specifies that one traverses the chain rule from inside to outside (that is, first compute dw_1/dx and then dw_2/dw_1 and at last dy/dw_2), while reverse accumulation has the traversal from outside to inside (first compute dy/dw_2 and then dw_2/dw_1 and at last dw_1/dx). More succinctly,

1. **forward accumulation** computes the recursive relation: $\frac{dw_i}{dx} = \frac{dw_i}{dw_{i-1}}\frac{dw_{i-1}}{dx}$ with $w_3 = y$, and,
2. **reverse accumulation** computes the recursive relation: $\frac{dy}{dw_i} = \frac{dy}{dw_{i+1}}\frac{dw_{i+1}}{dw_i}$ with $w_0 = x$.

Figure 9: Auto-differentiation as presented by Wikipedia.